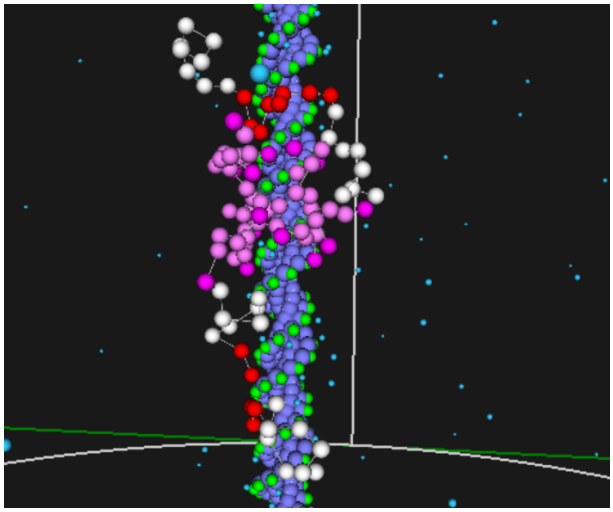
NTNU – Trondheim
Norwegian University of
Science and Technology

NTNU, Institutt for Fysikk

TMA4320 vår 2024 - Biofysikkprosjekt

Niels Henrik Aase og Pawel Sikorski

Monte Carlo-simuleringer av polymerfolding



Prosjektperiode: 24.01.24 - 06.02.24

Praktisk informasjon

Innleveringsfrist: Tirsdag 06.02.24, kl 23.59
Innleveringsplattform: Inspera (Se wikisiden til faget for mer info)
Språk: Dere kan velge om dere vil svare på engelsk eller norsk
Vurderingsansvarlig: Niels Henrik Aase, mail: `niels.h.aase@ntnu.no`
Innleveringsformat: Jupyter Notebook.
NB! Notebooken skal være kjørt ved innlevering. En enkel måte å sjekke om den er kjørt er å laste ned den endelige notebooken dere skal levere og sjekke at figurene fortsatt er der.

1 Introduksjon

Polymerer består av repeterende enheter som er bundet sammen av sterke kovalente bindinger. Disse underenhetene kalles monomerer, og vekselvirkningene mellom dem er det som gir formen til polymerer. Fagfeltet som jobber med dette, polymerfysikk, er svært aktivt, og det er flere grunner til dette. Den viktigste er at det kan brukes til å beskrive biopolymerer for eksempel som DNA, RNA, og proteiner. DNA (deoksyribonukleinsyre) brukes til å lagre arvestoffet til alle (kjente) levende organismer. Monomerene i DNA-kjeden kalles nukleotider, og disse nukleotidene er bundet sammen i to forskjellige tråder som sammen danner en dobbelhelliksstruktur. RNA (ribonukleinsyre) består også av nukleotider, men i de fleste organismer består RNA-molekylene av en enkelt tråd. I denne tråden danner nukleotidene kovalente bindinger med andre nukleotider i samme tråd, og RNA har derfor mye mer variert struktur enn DNA. Dobbelheliks og løkker kalt "hairpin loops" er vanlige strukturer for et RNA-molekyl.

Proteiner består av ett eller flere peptidkjeder, som er en rekke med aminosyrer. Rekkefølgen til aminosyrene kalles primærstrukturen til proteinet. Sekundærstrukturer dannes ved at deler av peptidkjedene folder seg i enten alfahelikser (spiraler), eller betaplater (flate former). Større proteiner kan ha mer komplekse foldinger som kommer av vekselvirkninger mellom to ulike områder av peptidkjeden, dette kalles tertiærstruktur. Dersom proteinet består av flere peptidkjeder som holdes sammen av disse vekselvirkningene, snakker vi om kvartærstruktur. Ikke alle proteiner har tertiær- og kvartærstruktur, og i dette prosjektet er det mest naturlig å klassifisere strukturen dere ser på som sekundærstrukturen til et protein.

Den tredimensjonale strukturen til et protein gir opphav til proteinets funksjonalitet. Et godt eksempel på dette er enzymer, som er en gruppe proteiner som setter igang og fremskynder kjemiske reaksjoner. Dette skjer ved at stoffene som inngår i den kjemiske reaksjonen binder seg til enzymet, som senker terskelen for at reaksjonen skal skje. Strukturen til enzymet fremstilles ofte som en puslespillbrikke med plass til disse stoffene, og strukturen er slik at det skal være mulig for stoffene å både binde seg og "slippe taket" igjen. Foldingen til proteinet er dermed essensiell for at det fungerer som det skal. Videre er feil i foldingen av proteiner knyttet til både Alzheimer's og Parkinson's sykdom, og oppførselen til disse misfoldede proteinene er et viktig forskningsområde innen biofysikk.

Å modellere og simulere folding av polymerer og proteiner har vært en av hovedbeskjeftigelsene innenfor numerisk biologi/biofysikk de siste tiårene. To av de mest brukte metodene i denne perioden er molekylærdynamikk, som tar utgangspunkt i litt forenklede modeller for proteiner/polymerer og bruker Newtons lover direkte til å se hvordan polymerene oppfører seg over tid, mens den andre metoden er bruk av storskala Monte Carlo-simuleringer. Vi skal bruke Monte Carlo i dette prosjektet, og kommer tilbake til detaljene senere, men det viktigste kjennetegnet er at den er *stokastisk*, ikke deterministisk. Det vil si at de tar i bruk tilfeldig utvelging for å få numeriske resultater. I Monte Carlo sees det ikke på utvikling over tid, men hva som er de mest sannsynlige konfigurasjonene (tilstandene) til polymerene.

Til tross for suksessen disse *klassiske* algoritmene har hatt, er de svært begrenset av størrelsen til polymeren og detaljnivået polymerene beskrives med. En type algoritme uten disse begrensingene er *kvantealgoritmer* som kjøres på kvantedatamaskiner. Selv om selve kvantealgoritmenene allerede for proteinfolding har blitt ganske sofistikerte [1], er maskinvaren ikke kommet langt nok enda til at dette er realistisk alternativ de neste årene. En annen moderne tilnærming til proteinfolding er å bruke dyp læring. Dette ble gjort i programmet AlphaFold [2] i 2018 og 2021, og predikerte strukturen til en rekke proteiner langt bedre enn konvensjonelle metoder.

I dette prosjektet skal dere selv skrive kode for å simulere polymerfysikk ved bruk av Monte Carlo-metoder. Polymerfysikk er progammeringsmessig krevende, som setter krav til hvor effektiv koden deres må være, spesielt når dere mot slutten av prosjektet skal forsøke å gjøre storskala simuleringer. Basert på resultatene fra simuleringene skal dere anslå de fysiske egenskapene til polymerer ved hjelp av enkle statistiske metoder.

2 Numerisk polymerfysikk

En polymer består av mange monomere som er bundet sammen av sterke kovalente bindinger. Til sammen danner disse monomerene en polymer. I dette prosjektet antar vi at de kovalente bindingene er så sterke at de aldri kan brytes. Det betyr at en monomer alltid skal være rett ved siden av naboene sine i polymeren. For å beskrive hvordan monomerene kan være rett ved siden av hverandre må vi først diskutere hva slags koordinatsystem vi bruker for å beskrive posisjonen til monomerene. Vi gjør to store forenklinger her. Den første forenklingen er at vi antar at polymerer (og tilhørende monomerer) bor i to dimensjoner [1] . Den andre forenklingen er at vi bruker et todimensjonalt diskret koordinatsystem slik at monomerene kun kan befinne seg på gitterpunktene i et kvadratisk gitter (Norsk og Engelsk fagbegrep: "oppdeling" og "grid"). For enkelthetens skyld definerer vi avstanden mellom to nærliggende gitterpunkter til å være 1. Vi kan da videre definere begrepet *nærmeste nabo* som et gitterpunkt som ligger rett til høyre/venstre eller rett over/under et annet punkt, altså at avstanden mellom

[1] Selv om valget av å holde oss i to dimensjoner her gjøres hovedsakelig for å ikke gjøre den numeriske implementasjonen av polymerrotasjoner enklere, så nevner vi at lavdimensjonale systemer er ikke en akademisk kuriositet. Det er blant annet relevant i moderne materialfysikk.

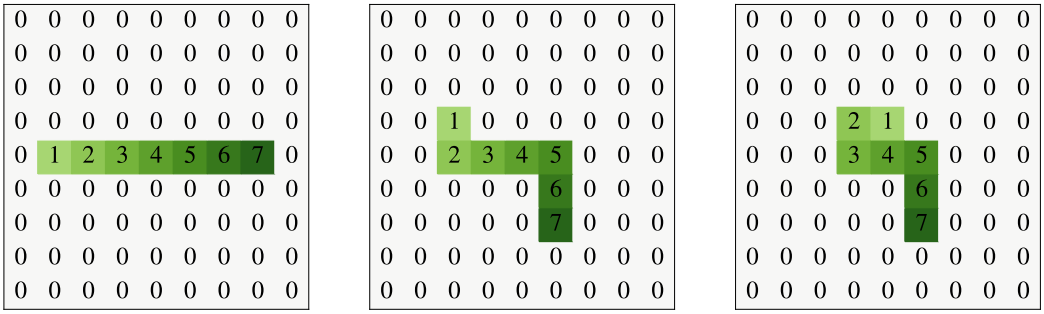


Figure 1: Tre todimensjonale polymerer. 0 representerer fraværet av en monomer, mens tallene representerer en monomer. Fargene på monomerene er inkludert for å gjøre figurene enklere å lese.

punktene er 1. Dette blir nyttig senere.

Med disse forenklingene kan vi definere hvordan en polymer bestående av N monomerer må se ut. Hvis vi nummererer hvert monomer fra $m = 1$ til $m = N$, slik at m er variabelen som representerer hver monomer, så definerer vi en polymer som en struktur der hver monomer m skal være nærmeste nabo med monomerene $m - 1$ og $m + 1$. Endemonomerene $m = 1$ og $m = N$ trenger kun å være nærmeste nabo med henholdsvis $m = 2$ og $m = N - 1$. Tre polymerer som oppfyller denne regelen er illustrert i Figur 1. Hvert monomer er markert med et tall og en farge, og fraværet av en monomer representeres av 0. Polymeren til venstre er helt rett og oppfyller åpenbart definisjonen på en polymer. Polymeren i midten oppfyller også definisjonen, selv om det har blitt rotert én gang rundt monomer 2 med klokken, og én gang rundt monomer 5, også der med klokken. Til slutt har vi polymeren til høyre som har opplevd de samme rotasjonene som den i midten, men i tillegg har rotert om monomer 3 med klokken. Detaljene rundt rotasjonene kommer vi tilbake til snart.

I tillegg til de sterke kovalente bindingene som holder polymerer sammen, vil det være andre svakere vekselvirkninger som er med på å bestemme formen til en polymer. Det er for eksempel svake hydrogenbindinger som er med på å gi DNA sin karakteristiske heliksform. I dette prosjektet antar vi for enkelhetens skyld at de kovalente bindingene kun er ansvarlige for at polymeren aldri blir ødelagt. Utover dette bestemmer de ikke formen til polymeren, og de bidrar heller ikke til den totale energien til polymeren. De svake vekselvirkningene virker kun mellom monomerer som ikke allerede er bundet av kovalente bindinger, og er da de eneste vekselvirkningene som bidrar til polymerens energi og form. Vekselvirkningene har svært kort rekkevidde, så vi antar at monomerene må være nærmeste naboer for at de skal være større enn null. For å oppsummere:

- Monomer m opplever ikke vekselvirkninger fra $m - 1$ og $m + 1$ (og heller ikke fra seg selv m).
- For at monomerene m og n (der $n \neq m, n \neq m + 1, n = m - 1$) skal vekselvirke må de være nærmeste naboer.

Med reglene ovenfor ser vi at de to første polymerene i Figur 1 begge har null energi siden det ikke opplever noen svake vekselvirkninger. I polymeren til høyre er det én svak vekselvirkning siden monomer 1 og 4 er nærmeste naboer.

Mer matematisk kan vi da formulere uttrykket for energien E til en polymer som

$$E = \frac{1}{2}\sum_{m,n} V_{mn} b_{mn}, \tag{1}$$

der vi har introdusert $N \times N$-matrisen V og operatoren b_{mn}. Den symmetriske matrisen V inneholder styrken til vekselvirkningene, og vi kan ta hensyn til det første kulepunktet ovenfor ved å kreve at både diagonalen, overdiagonalen og underdiagonalen til V alltid er null. Det vil si at $V_{mm} = 0$, $V_{m,m+1} = 0$ og $V_{m+1,m} = 0$. Operatoren b_{mn} er definert som 1 dersom monomer m og n er nærmeste naboer, ellers er den null. Den tar dermed hensyn til det andre kulepunktet ovenfor. Summen for både m og n går fra 1 til N, slik at vi må normalisere med en faktor $1/2$ siden vi da teller alle vekselvirkningene to ganger.

2.1 Numerisk representasjon av polymerer

I figur 1 er polymeren, i tillegg til å befinne seg på et gitter, også lagret som et gitter. Det vil si at hele gitteret, inklusive alle null-elementene, er lagret som en matrise (to-dimensjonal array) av størrelse $M \times M$. M må oppfylle $M \geq N$ for at en helt rett polymer skal få plass på gitteret. Utover at M skal oppfylle $M \geq N$, så må dere velge en hensiktsmessig verdi for M selv.

Å lagre hele gitteret er en av minst to måter å gjøre det på. Et problem med denne representasjonen er den overflødige informasjonen som lagres. Vi er bare interessert i posisjonen til N monomere, men ved å lagre hele gitteret lagrer vi M^2 verdier, der de fleste av verdiene bare er nullere. Den andre måten, som vi her kun hinter vagt til, kondenserer den samme informasjonen som i gitterene i Figur 1 og krever bare lagring av en array med dimensjon $N \times 2$. Kjøretidsforbedringen til denne metoden er betydelig, men vi understreker at dere fint kan få full uttelling på oppgavene uavhengig av hvilken metode dere velger å bruke.

2.2 Rotasjoner

Siden monomerene er definert på et kvadratisk gitter så er de interne vinklene som dannes alltid multiplum av 90 grader. Dette gjenspeiles i hvordan vi definerer en rotasjon på en polymer. Den defineres alltid som en rotasjon på 90 grader, enten med eller mot klokken. For hver rotasjon trenger vi en monomer som vi skal rotere rundt. La oss kalle denne monomeren for m_{rot}. Hvis vi deler polymeren i to deler om m_{rot}, $m < m_{\text{rot}}$ og $m > m_{\text{rot}}$, så skal kun den den ene delen rotere om m_{rot}, den andre forblir i ro. I dette prosjektet kan dere alltid velge hvilken del av polymeren som skal rotere. Det er da nyttig å alltid la den korteste delen roterere og la den lengste delen forbli i ro. Dette valget gjør at

monomeren i midten av polymeren alltid forblir i ro [2] , som også gjør at polymeren aldri beveger seg ut av gitteret. Dersom dere (mot formodning) tillater rotasjon av både den korte og lengste delen av polymeren kan dere bruke periodiske grensebetingelser hvis det blir nødvendig.

Merk at enkelte rotasjoner vil føre til at polymeren blir ødelagt, altså at det bryter definisjonen av en polymer. Dette ville for eksempel om man forsøkte å rotere polymeren til høyre i Figur 1 med klokken om monomer 3. Da ville to monomere okkupert samme gitterpunkt, som bryter med definisjonen. Dere få en konkret sjekkliste om hva som definerer en intakt polymer senere i oppgaveteksten.

2.3 Slumptall og stokastiske simuleringer

I dette prosjektet skal dere simulere en stokastisk prosess. Dette forutsetter at man kan generere tilfeldige tall. En datamaskin har egentlig ikke kapasitet til å generere *helt* tilfeldige tall (norsk fagbegrep: ekte slumptall) siden den er grunnleggende deterministisk [3]. Man kan i stedet generere uekte slumptall/pseudotilfeldige tall (engelsk: pseudorandom numbers) ved å bruke slumptallgeneratorer, som i de aller fleste praktiske formål fungerer som ekte slumptall.

Simuleringene dere skal gjøre i dette prosjektet bruker uekte slumptall. I den første simuleringen dere skal gjøre simuleres bevegelsen til en polymer som alltid oppfyller reglene vi beskrev tidligere, men der bevegelsesmønsteret er helt tilfeldig. Pseudokoden for en slik simulering er i Algoritme 1, og forutsetter at man fått implementert den grunnleggende polymerfysikken som er forklart tidligere.

Algorithm 1 Pseudokode for å tilfeldig rotere en polymer bestående av N monomere N_s ganger. Den returnerer sluttposisjonen til polymeren og antall vellykkede rotasjoner.

```
counter ← 1
polymer ← rett horisontal polymer
for i = 1 to N_s do
Velg en tilfeldig monomer å forsøke å rotere om
Velg tilfeldig om det skal forsøkes å rotere med eller mot klokken.
    twisted_polymer ← twist(polymer, random_monomer, twist_clockwise)
    if check_if_legal_twist(twisted_polymer) then
        counter = counter + 1
        polymer ← twisted_polymer
    end if
end for
return polymer, counter
```

I Python er den enkleste måten å genere pseudotilfeldige tall å bruke `numpy.random`-

[2] Merk at monomeren i midten ikke har en unik definisjon når N er et partall. For N partall må dere kanskje passe litt ekstra på hvilken ende dere velger å rotere, pass på at dere er konsekvente.

[3] Ekte slumptall kan genereres ved å bruke målinger fra for eksempel radioaktivitet eller andre stokastiske fysiske prosesser.

biblioteket. I dette prosjektet kommer dere i hovedsak til å bruke to funksjoner fra dette biblioteket:

- `numpy.random.uniform(low, high, size)`: Genererer uniformt fordelte tilfeldige reelle tall $\in (low, high)$ og returner en `numpy`-array med dimensjoner angitt i `size`.
- `numpy.random.randint(low, high, size)`: Genererer uniformt fordelte tilfeldige heltall $\in [low, high)$ (merk det delvis åpne intervallet) og returner en `numpy`-array med dimensjoner angitt i `size`.

3 Monte Carlo-metoder

Monte Carlo-metoder er et begrep som dekker en rekke algoritmer. Det viktigste kjennetegnet på disse algoritmene er at de bruker tilfeldig utvelging. En av de mest brukte Monte Carlo-metodene i fysikk er Metropolisalgoritmen. Den har blitt (og blir fortsatt!) anvendt på en rekke problemer i statistisk fysikk og brukes ofte som gullstandarden andre algoritmer og teknikker sammenlignes med.

Før vi forklarer Metropolisalgoritmen nærmere, la oss først motivere hvorfor vi trenger den. Et fysisk system kan befinne seg i svært mange tilstander, hvor vi lar S beskrive en generell tilstand i systemet. I polymersystemet vi ser på i dette prosjektet representerer S samlingen av alle posisjonskoordinatene til hver monomer som til sammen danner en polymer. Videre ønsker vi nå å se på egenskap til systemet x som vi vet hvordan vi kan beregne fra en tilstand S. Det vil si at vi kan beregne $x(S)$ for en gitt tilstand. Som et konkret eksempel, hvis vi ønsker å beregne diameteren til en polymer, så kan vi alltid finne den største avstanden fra en monomer til en annen direkte fra tilstanden S. En grunnpilar i både statistisk fysikk og kvantemekanikk er at størrelser vi måler i den virkelige verden, *makroskopiske* størrelser, er forventningsverdien til den tilsvarende *mikroskopiske* størrelsen. Det vil si at når vi måler x, så måler vi egentlig $\langle x \rangle$, altså forventningsverdien til x. Hvis vi snur på det og sier at vi har en mikroskopisk modell, så er den størrelsen vi er mest interessert i $\langle x \rangle$ siden det er den vi kan måle. $\langle x \rangle$ er gitt ved

$$\langle x \rangle = \sum_{S \in \text{ tilstander}} P(S)x(S), \tag{2}$$

hvor $P(S)$ er sannsynligheten for at systemet befinner seg i tilstand S. Summen her er over alle mulige tilstander systemet kan befinne seg i. Og her ligger også problemet. Et mikroskopisk system har utrolig mange tilstander det kan befinne seg i, altfor mange til at vi kan bruke ligning (2). For eksempel har et system med 40 kronestykker, over én billion (2^{40}) tilstander, der hver tilstand er en unik konfigurasjon/følge med 40 kron/mynt. For polymer-systemet i forrige seksjon er situasjonen enda verre (Tenk hvor mange forskjellige polymerer vi kan lage med bare 20 monomerer).

I ligning (2) introduserte vi $P(S)$ som sannsynligheten for at systemet er i tilstand S. Fra svært grunnleggende prinsipper kan man utlede at denne sannsynligheten kun er avhenging av temperaturen til systemet, samt hvor energetisk gunstig tilstanden er.

Dette er grunnen til at energi-funksjonen er den viktigste egenskapen til et system [4] . Formen til $P(S)$ er svært enkel, den er

$$P(S) = \frac{1}{Z} \mathrm{e}^{-\beta E(S)}, \tag{3}$$

hvor Z er en normaliseringsfaktor, kalt partisjonsfunksjonen (som har langt flere og dypere egenskaper enn bare å være en normaliseringsfaktor, detaljer vi ikke går inn på her), og β er invers proporsjonal med temperaturen

$$\beta = \frac{1}{k_{\mathrm{B}} T}, \tag{4}$$

der $k_{\mathrm{B}} = 1.38 \cdot 10^{-23}$ JK^{-1} er Boltzmanns konstant. **NB!** I dette prosjektet skal dere alltid bruke Kelvin som enhet på temperatur.

Når et system har svært mange tilstander, er Z vanskelig å beregne, akkurat som ligning (2). Heldigvis kan vi enkelt utlede et uttrykk for forholdet mellom sannsynligheten for å befinne seg i tilstand S_n og S_i. Direkte innsatt i ligning (3) får vi da

$$\frac{f(P_n)}{f(P_i)} = \mathrm{e}^{-\beta(E(S_n) - E(S_i))} = \mathrm{e}^{-\beta \Delta E}, \tag{5}$$

der

$$\Delta E = E(S_n) - E(S_i). \tag{6}$$

For å finne sannsynlighetsforholdet mellom to tilstander trenger vi dermed kun å se på endringen i energi ΔE. Vi trenger ikke beregne Z siden den kanselleres.

3.1 Metropolisalgoritmen

Metropolisalgortimen brukes for å få bukt med alle de mulige tilstandene et system kan befinne seg i, ved å finne et representativt utvalg av tilstander. Den er ikke innom alle tilstander, men tilstandene den er innom danner et godt nok utvalg, gitt at det simuleres lenge nok. Måten algoritmen virker på kan forklares fra ligning (3) og (5). La oss se på et system som starter i tilstanden S_i. Igjen, for å være konkrete ser vi på polymersystemet fra seksjon 2. Vi velger en tilfeldig monomer å rotere polymeren om (også tilfeldig om vi roterer med eller mot klokken) som vi antar gir et inntakt polymer. Den roterte versjonen av S_i kaller vi S_n. I Metropolisalgoritmen er det to måter vi faktisk lar tilstanden endre seg fra S_i til S_n:

1. Som vi ser fra ligning (3), ønsker et system å minimere energien sin. Så hvis $\Delta E < 0$, så endres tilstanden til systemet fra S_i til S_n.

2. Selv om systemet ønsker å minimere energien sin, så kan fortsatt energetiske ugunstige endringer skje ($\Delta E > 0$) på grunn av termiske fluktuasjoner. Jo høyere

[4] I kvantemekanikk, klassisk mekanikk og feltteori er energifunksjonen ofte den *eneste* funksjonen vi trenger for å fullstendig beskrive et system. Mer om dette i fag som TFY4205, TFY4345 og FY3464.

temperatur (som gir sterkere termiske fluktuasjoner), jo mer sannsynlig blir dette. Måten vi tar hensyn til denne muligheten er å sammenligne sannsynlighetsforholdet i ligning (5), $\exp(-\beta\Delta E)$, med et tilfeldig reellt tall mellom 0 og 1 som vi kaller p. Hvis $\exp(-\beta\Delta E) > p$, så endrer vi tilstanden til systemet fra S_i til S_n, til tross for at endringen er energetisk ugunstig. Denne regelen gjør at systemet kan utforske flere tilstander, ikke bare de som har lavest energi. Med høyere temperatur utforsker systemet flere tilstander. Hvis $\exp(-\beta\Delta E) < p$ endrer vi ikke tilstanden, og lar systemet være i S_i.

I dette prosjektet definerer vi prosessen ovenfor som ett Monte Carlo-steg som vi kaller t. Merk at, til tross for valget av variabel, t ikke skal tolkes som tid, vanlige Monte Carlo-metoder brukes ikke til å se på dynamikk. Storskala Monte Carlo-simuleringer kan, litt forenklet, sees på som å gjøre svært mange Monte Carlo-steg. Det er enkelte statistiske aspekter ved Monte Carlo-metoder som vi ikke går videre inn på her eller i oppgavene, men vi refererer til [3] for mer detaljer om dette, samt et lite notat i slutten av prosjektbeskrivelsen for den interesserte leser.

Før dere starter på oppgave 2, kan det være lurt å tenke gjennom hvordan en simulering ved høy temperatur vil skille seg fra en med lav temperatur. Forskjellen bestemmes av ulikheten $\exp(-\beta\Delta E) > p$, så tenk gjennom hvordan den avhenger av temperatur. Ved veldig lave temperaturer, hva skjer dersom systemet havner i et lokalt energiminimum?

3.2 Pseudokode for Metropolisalgoritmen

I Algoritme 2 har vi gitt dere pseudokode for hvordan en Monte Carlo-simulering i dette prosjektet kan se ut. I oppgave 1 vil dere lage all nødvendig polymer-funksjonalitet før dere skal implementere og bruke algoritme 2 i oppgave 2.
NB! Her definerer vi et Monte Carlo-steg der den potensielt nye tilstanden, S_n, må være en lovlig tilstand, altså at den oppfyller reglene for en polymer. Hvis S_n er ulovlig, så forkastes tilstanden, og vi teller ikke dette som et Monte Carlo-steg.

Algorithm 2 Pseudokode for Monte Carlo simulering av energien til en polymer med vekselvirkningsmatrise V

```
polymer, N_s, V og T er inputparametre
E_array ← N_s array med nullere
E ← calculate_energy(polymer, V)
i ← 1
while i < N_s do
Velg en tilfeldig monomer å forsøke å rotere om
Velg tilfeldig om det skal forsøkes å rotere med eller mot klokken.
    twisted_polymer ← twist(polymer, random_monomer, twist_clockwise)
    if check_if_legal_twist(twisted_polymer) then
        i ← i + 1
        E_new ← calculate_energy(twisted_polymer, V)
        if E_new < E then
            polymer ← twisted_polymer
            E ← E_new
        else if RAND(0, 1) < exp(−β(E_new − E)) then
            polymer ← twisted_polymer
            E ← E_new
        end if
        E_array(i) ← E
    end if
end while
return polymer, E_array
```

4 Oppgaver

Oppgavene er nedenfor. I den endelige besvarelsen (i Jupyter Notebook) kan dere referere direkte til oppgavenummer (f. eks **1e)**). Pass likevel på at besvarelsen ikke blir en oppramsing av svarene på oppgavene, den skal være en sammenhengende tekst. Dere skal ha med en kort konklusjon og en introduksjon som skal kunne leses uten å ha lest prosjektbeskrivelsen. Ellers kan dere anta at leseren har god kjennskap til temaet og prosjektbeskrivelsen.

Det er flere måter å strukturere notebooken på, men vi anbefaler å flette sammen koden, resultatene og rapporten. Som et konkret eksempel kan dere først kort motivere og forklare hva dere gjør i oppgave **1a)** og **1b)** i vanlig tekstformat, så ha en kodeblokk som inneholder selve implementasjonen dere blir bedt om i **1b)**. Alle plott skal forklares/diskuteres og skal ha fysiske enheter der det er nødvendig. Koden deres skal være enkel å lese og forståelig med informative variabelnavn. Bruk gjerne kommentarer. Godt dokumentert og strukturert kode vil trekke besvarelsen i positiv retning. Kjøretiden på koden deres vil variere (mye) avhenging av hvordan dere løser oppgavene, men vil ikke påvirke endelig karakter utover at rask/lur/godt strukturert kode vil telle i positiv retning. Det er fint mulig å få full uttelling med tregere kode, spesielt om det forklares godt i de deloppgavene der dere eksplisitt blir bedt om å evaluere kodehastigheten.

Oppgave 1: Grunnleggende polymer-funksjonalitet

1a) Forklar hvordan dere vil representere en polymer numerisk, altså hvordan dere lagrer posisjonen til hvert monomer. Her er det minst to lure valg som forklart i i seksjon 2.1, der det ene (og minst effektive av de to) er brukt tidligere i prosjektbeskrivelsen.

1b) Bruk datastrukturen dere valgte i **1a)** og lag en funksjon som tar inn N og returnerer en polymer av størrelse N formet som en rett horisontal linje.

1c) Basert på reglene som styrer polymerene i seksjon 2, hva skjer med energien til polymerene dersom de roteres om et endepunkt? I algoritmene 1 og 2, står det at dere skal velge et tilfeldig monomer å rotere om i hvert steg. Vurder om det er hensiktsmessig å inkludere endemonomerene i utvalget den tilfeldige monomeren trekkes fra.

1d) Lag en funksjon som tar inn en polymer og illustrerer det på en hensiktsmessig måte. Hver monomer skal kunne unikt identifiseres (altså skal ikke alle monomerene representeres av samme farge) og enkelt skilles fra bakgrunnen (der det ikke er monomerer). Bruk funksjonen til å visualisere den rette polymeren fra oppgave **1b)**.
Tips: Hvis dere bruker `plt.pcolormesh` til å visualisere polymeren så vil dere legge merke til at polymeren er speilet om x-aksen. Dette kan dere endre på ved å bruke *slicing* slik at radverdiene kommer i motsatt rekkefølge.

1e) Lag en funksjon som tar inn en polymer og polymerens størrelse N. Funksjonen skal returnere `True` hvis det er intakt. En intakt polymer av størrelse N oppfyller kravene:

1. Det har N monomerer.
2. Hver monomer representeres av et unikt heltall $\in [1, N]$. Det vil si at ingen monomerer representeres av det samme tallet som en annen monomer.
3. En monomer representert av tallet m skal være *nærmeste nabo* med monomerene representert av tallene $m-1$ og $m+1$. Endemonomerene $m = 1$ og $m = N$ trenger kun å være nærmeste nabo med henholdsvis $m = 2$ og $m = N - 1$.

1f) Lag en funksjon som tar inn en polymer, en koordinat der det befinner seg en monomer, og en boolsk variabel som sier om rotasjonen er med eller mot klokken. Funksjonen skal returnere en polymer som er rotert med/mot klokken om koordinaten, i henhold til forklaringen av rotasjoner i seksjon 2.2.
Tips: Denne funksjonen må ikke alltid returnere en intakt polymer. Den skal senere alltid brukes sammen med funksjonen i **1d)** som sjekker om polymeren er inntakt. **NB!** Dette er med all sannsynlighet den mest krevende deloppgaven i hele prosjektet, så ikke fortvil om dere bruker mye tid på den. Likevel, det er ikke meningen at dere skal kjøre dere helt fast, så kom i øvingstimene for å få hjelp av flinke studasser.

1g) Lag en funksjon som tar inn N og N_s, der N_s er antall ganger en polymer skal forsøke å utføre en rotasjon, og implementer algoritme 1. Kjør funksjonen først med $N = 15$ og $N_s = 4$ og så med $N = 15$ og $N_s = 1000$. Lagre sluttposisjonen til polymeren i hver simulering ved å bruke for eksempel `np.savez`. Illustrer og sammenlign sluttposisjonen til begge simuleringene.

Av og til kreves store dataressurser for å kjøre beregninger/simuleringer som gir tilstrekkelig nøyaktighet. En slike type ressurs er superdatamaskiner som vi har på NTNU. Likevel, så kan man i mange tilfeller heller skrive bedre/raskere kode enn å bruke den største datamaskinen man har tilgjengelig til å kjøre ineffektiv kode. Dette er hensiktsmessig av flere grunner, blant annet er bruk av superdatamaskiner både dyrt og svært energikrevende.

Dere har allerede lært litt om hvordan man kan forbedre kjøretid i første læringsbolk av faget, men her en veldig kort oppsummering: Såkalte høynivåprogammeringsspråk (Python, Matlab) er mer fleksible og typisk lettere å bruke enn raskere (og vanskeligere) kompilerte språk som C++ og Fortran. Heldigvis har Python en rekke funksjonaliteter som gjør at man kan oppnå sammenlignbar hastighet som med kompilerte språk i enkelte tilfeller. Den enkleste måten er å bruke `numpy`-funksjoner så ofte som mulig. Et annet alternativ er å bruke biblioteket `numba`, som kompilerer funksjoner på forhånd[5].

[5] Å komme i gang med `numba` er litt krevende, det er endel ting man ikke kan gjøre (Eksempelvis kan man ikke lage tomme lister og man må ofte definere datatyper) og feilmeldingene man får er vanskelige

Flere av deloppgavene i dette prosjektet vil ha litt kjøretid. Avhengig av koden deres kan man forvente alt fra et par minutter til én-to timer. Mye av kjøretiden vil ligge i å utføre rotasjoner på polymerer, det vil si at kjøretiden ligger nesten kun i funksjonaliteten dere skrev i oppgave **1e)** (og delvis **1d)**). Derfor vil dere kunne spare mye tid på å gjøre funksjonaliteten i **1e)** mer effektiv, som er neste deloppgave. I Jupyter Notebook kan dere bruke kommandoen `%timeit` til å evaluere hvor lang tid en kodebolk bruker på å kjøre.

1h) Basert på det dere har lært hittil om kodehastighet, forsøk å øke hastigheten til funksjonen dere skrev i **1f)** og/eller **1e)**. Bruk funksjonen i **1g)** med $N = 15$ og $N_s = 1000$ sammen med kommandoen `%timeit` for å teste hvor mye raskere den nye funksjonen kjører. Hvis dere lykkes med å øke hastigheten, forklar hvorfor koden nå er raskere. Hvis ikke, kan dere likevel få full uttelling dersom dere diagnostiser og drøfter hvor kjøretiden kunne blitt forbedret. Drøft også hvordan kjøretiden (med implementasjonen deres) skalerer med størrelsen på på polymeren.

1i) Bruk funksjonen i **1g)** til å plotte prosentandelen gyldige rotasjoner en polymer utsettes som funksjon av N. Velg selv hvor mange N-verdier dere vil ta med. Bruk først $N_s = 1000$, men diskuter også hvordan andre valg av N_s påvirker grafen.
Tips: Det kan være diskusjonen av N_s sin rolle blir enklere om dere eksperimenterer med forskjellige valg av N_s. Inkluder gjerne i så fall flere grafer i figuren.

1j) Lag en funksjon som beregner energien til en polymer ved hjelp av ligning (1). Funksjonen må ta inn både polymeren og matrisen V. For en V der alle de ikke-null elementene er lik -1, beregn energien til de to polymerene i oppgave **1f)**. Kommenter resultatet, hvilket av polymerene har lavest energi? Er dette forventet?

Oppgave 2: Monte Carlo-simuleringer

Etter oppgave 1 har dere implementert all nødvendig polymerfunksjonalitet slik at dere nå kan begynne med Monte Carlo-simuleringer. Før vi starter på disse oppgavene, la oss kort dvele med hva slags systemer vi skal se nærmere på. Polymerer består av repeterende enheter (monomerer) som er bundet sammen av sterke kovalente bindinger. Eksempeler på polymerer er proteiner og DNA. I proteiner er den repeterende enheten aminosyrer, som vekselvirker gjennom svake hydrogenbindinger. Styrken på disse bindingene varierer avhenging av hvilke aminosyrer som er involvert og omgivelsene til proteinene. Verdiene dere vil blir bedt om å bruke er, innenfor én størrelsesorden, realistiske for å beskrive hvordan aminosyrene til et protein vekselvirker i en vannløsning.

Til slutt, for ordens skyld: Dere blir bedt om å studere et temperaturintervall som er langt større enn der mesteparten av livet på jorda trives. Dette er hovedsakelig for

å tolke. Det er likevel et utrolig nyttig verktøy i Python, så det kan være verdt å forsøke og bruke det i dette prosjektet, men det er helt frivillig.

å gjøre temperaturavhengigheten til polymerens størrelse og energi tydeligere. Denne avhengigheten er også observerbar for mer realistiske temperaturer [4].

2a) Implementer Metropolisalgoritmen som beskrevet i algoritme 2. Funksjonen skal ta inn utgangsposisjonen til polymeren som simuleres, samt vekselvirkningsmatrisen V, antall rotasjoner N_s som skal utføres i simuleringen og temperaturen T.

2b) Kjør to simuleringer med funksjonen dere skrev i **2a)**. Input til simuleringene skal være like med unntak av temperatur. Velg en verdi for N mellom 20 og 35, og bruk en rett horisontal polymer som startkonfigurasjon i begge simuleringene. Bruk $N_s = 5000$ og en V der alle de ikke-null elementene er lik $-4.0 \cdot 10^{-21}$ J. I den første simuleringen skal dere bruke en valgfri lavere temperatur i intervallet $50\ \text{K} \leq T_{\text{lav}} \leq 100\ \text{K}$, mens i den den andre skal dere bruke en høyere temperatur $293\ \text{K} \leq T_{\text{hoy}} \leq 373\ \text{K}$. Plott energien som funksjon av antall Monte Carlo-steg, t. Sammenlign grafene og forklar det dere ser. Hvilken av simuleringene oppnår den laveste energien i løpet av simuleringen? Forklar hvorfor.
Hint: Her kan det være relevant å knytte drøftingen deres til konseptet av lokale energiminima.

2c) Illustrer sluttposisjonen til de to polymerene fra simuleringen i **2b)** ved siden av hverandre. Kommenter resultatet i lys av diskusjonen deres i **2b)**.

Hittil har dere kun sett på systemer der alle de ikke-null-verdiene i V har samme verdi. I virkeligheten er ikke dette tilfellet, variasjon i matriselementene til V er nettopp det som er med på å gi proteiner og polymerer sine unike strukturer. Et sentralt postulat innen biokjemi har lenge vært at funksjonen til et protein er avhenging av strukturen, som gjør at et proteins påvirkning kan endres dersom V, og dermed også strukturen, endres. Dere skal nå selv forsøke å endre strukturen en polymer ved å endre V.

2d) Lag nå selv en V der det eneste kravet er at den fortsatt er symmetrisk. Basert på valget deres, hvordan forventer dere at en polymer med deres V folder seg? Kjør en passende simulering der dere tester om polymeren folder seg på måten dere forventer. Illustrer sluttposisjonen til polymeren og drøft resultatet.
Tips: Hvis dere sliter med å komme på en interessant problemstilling kan dere for eksempel vurdere å skalere enkelte av matriseelementene med en faktor 100. Hvordan vil dette påvirke strukturen til polymeren?

I de neste deloppgavene skal dere anslå forventningsverdien til energien og diameteren til polymerer, samt standardavviket til energien σ_E. Bruk gjennomsnittet og det empiriske standardavviket til arrayen dere lagrer i Monte Carlo-funksjonen deres (Eksempelvis `E_array` i Algoritme 2) for å anslå henholdsvis forventingsverdier og standardavvik.

2e) Beregn $\langle E \rangle$ og σ_E som funksjon av T i temperaturintervallet $[10, 1000]$ K, med 30 K mellom hver temperatur. Når dere beregner $\langle E \rangle$ og σ_E, ekskluder de første 1000 stegene fra simuleringene. Start med en rett horisontalt polymer hver gang dere starter en ny simulering. Velg en verdi for N mellom 25 og 35, velg N_s selv, og bruk samme V som beskrevet i **2b)**. N, N_s og V skal ikke endre seg i løpet av simuleringene. Lag et plot med $\langle E \rangle$ som funksjon av T som også viser standardavviket σ_E som usikkerhetstolper (Engelsk: Error bars) ved å bruke `plt.errorbar`. Diskuter plottet og drøft temperaturavhengigheten til $\langle E \rangle$ og σ_E. Hvordan påvirker valget deres av N_s temperaturavhengigheten til $\langle E \rangle$?

2f) Velg en av de laveste temperaturene dere brukte i **2e)** og gjør 10 nye simuleringer med denne temperaturen og ellers de samme input-parametrene som i **2e)**. Plott $E(t)$ fra hver simulering i én figur og forklar det dere ser. Hvordan og hvorfor kan dette brukes til å bedre anslå energien til en polymer med lavere temperatur?

I de to siste deloppgavene skal dere se på diameteren til polymeren, d. Vi definerer d som den største Euklidiske avstanden mellom to monomerer i polymeren. d er en egenskap man kan måle på en lab, slik at man kan få samsvar mellom et ekte eksperiment og resultatene fra en numerisk modell, gitt at de frie parameterne i modellen, altså verdiene i V, er fornuftige. På denne måten kan man videre jobbe seg bakover og finne en realistisk V som styrer polymeren på et mikroskopisk nivå.

2g) Implementer algoritme 2 der dere nå også beregner d i hvert steg. Beregn $\langle d \rangle(T)$ på samme måte og for samme temperaturer som i **2e)**. Velg igjen en verdi for N mellom 25 og 35 og bruk nå en V der de ikke-null matriseelementene er valgt tilfeldig i intervallet $[-6 \cdot 10^{-21}, -2 \cdot 10^{-21}]$ J, der V fortsatt må være symmetrisk. Velg en så stor verdi for N_s som kjøretiden deres tillater, eventuelt en så stor N_s som dere anser som tilstrekkelig. Oppgi hvor lang kjøretid alle simuleringene til sammen tok. V, N og N_s skal være de samme i alle simuleringene, mens T varierer. Plott $\langle d \rangle(T)$ og diskuter plottet. Hvordan henger energien til polymerene sammen med d?

2h) Bruk samme V, N_s og N som i **2g)** i alle simuleringene i denne deloppgaven. Bruk også samme temperaturverdier som tidligere, men start nå med den høyeste temperaturen og iterer nedover. I tillegg skal dere nå ikke tilbakestille polymeren til horisontal posisjon når dere endrer temperatur slik at sluttposisjon ved en temperatur blir startposisjonen til neste temperatur. Hva slags (enkel) fysisk prosess tilsvarer dette? Plott $\langle d \rangle(T)$ fra disse simuleringene og $\langle d \rangle(T)$ fra **2g)** i samme figur. Sammenlign måleseriene i lys av de to forskjellige metodene som ble brukt i simuleringene. Drøft hvilken av metodene som egner seg best for å se på både lav- og høytemperaturoppførselen til en polymer.

Slutt på oppgaver.

Postskriptum: Statistiske aspekter ved Monte Carlo-metoder

NB! Dette er kun ment for å utbrodere litt om de statistiske aspektene ved prosjektet som vi ikke går inn på i oppgavene. Det er med andre ord ikke nødvendig å diskutere eller implementere noe av dette i den endelige besvarelsen dere og er derfor kun for interesserte lesere.

Monte Carlo-simuleringer brukes som regel i statistikk fysikk til å beregne en fysisk egenskap vi er interessert i som vi ikke klarer å beregne direkte. Direkte beregning skalerer ofte svært raskt med systemets størrelse, som diskuterert i seksjon 3. Kjøretiden som trengs for å få nøyaktige anslag av fysiske egenskaper vokser også med Monte Carlo, men den vokser saktere enn for eksempel molekylærdynamikk. Dette lar oss se på større systemer med Monte Carlo-metoder.

I oppgave **2e)** ba vi dere om å ekskludere de første 1000 stegene fra simuleringen. Dette skyldes at fysiske systemer ofte trenger litt tid fra de initialiseres i en tilfeldig starttilstand til de havner i en tilstand som er mer representativ for systemet ved en gitt temperatur. Tiden dette tar kalles ofte likevektstiden, t_{equil}, og vokser med størrelsen til systemet. Den henger også sammen med temperaturen til systemet. Eksempelvis eksploderer den i nærheten av *faseoverganger*, som er problematisk siden det ofte er nettopp disse vi ønsker å studere nærmere numerisk. Siden Monte Carlo-simuleringer er stokastiske, vil også t_{equil} variere fra simulering til simulering, derfor er det ofte lurt å overestimere t_{equil} slik at man er sikker på at systemet faktisk har nådd likevekt før man begynner med målinger av den egenskapen man er interessert i.

Når man har ventet t_{equil} steg, er det klart til å ta målingene som trengs for å anslå $\langle x \rangle$. I prosjektet gjøres dette ved å beregne x (som her representerer en vilkårlig fysisk egenskap, i prosjektet beregner dere E og d) for tilstanden S systemet er i ved forskjellige t, før man til slutt tar gjennomsnittet av alle målingene. Dette er tilstrekkelig i akkurat dette prosjektet, men det er egentlig overflødig å beregne x ved $t = t_{\text{equil}} + 1$ hvis det allerede er blitt beregnet ved $t = t_{\text{equil}}$. Hvorfor? Fra statistikk vet vi at gjennomsnittet til en stokastisk variabel er en god estimator for den sanne verdien til forventningsverdien til variabelen, men dette er kun sant hvis gjennomsnittet beregnes fra *uavhengige* stokastiske målinger. Åpenbart er ikke $x(S(t = t_{\text{equil}}))$ og $x(S(t = t_{\text{equil}} + 1))$ uavhengige; $S(t = t_{\text{equil}})$ og $S(t = t_{\text{equil}+1})$ skilles kun av en rotasjon. Vi kan fikse dette ved å vente t_r steg mellom hver måling, slik at vi faktisk bruker uavhengige målinger når vi beregner gjennomsnittet. Man kan optimere hvor lenge man bør vente mellom målinger, se [3] for detaljer, men dette går langt forbi pensum i dette prosjektet.

Til slutt, som dere også husker fra statistikk, så blir estimatoren $\bar{x}$ (gjennomsnittet) bedre når vi øker antall målinger for å beregne $\bar{x}$. Antall målinger kaller vi n og jo høyere nøyaktighet vi vil ha på anslagene våre for $\langle x \rangle$, jo høyere verdi for n trengs. Dermed blir det totale antall Monte Carlo steg man trenger $N_s = t_{\text{equil}} + t_r n$. Forskjellige fysiske systemer vil kreve forskjellige valg av hver av variablene t_{equil}, t_r og n, men det går vi ikke inn mer på her.

References

[1] A. Robert, P. K. Barkoutsos, S. Woerner, and I. Tavernelli, "Resource-efficient quantum algorithm for protein folding," npj Quantum Information **7**, 38 (2021).

[2] J. Jumper et al., "Highly accurate protein structure prediction with AlphaFold," Nature **596**, 583–589 (2021).

[3] M. Newman and G. Barkema, *Monte Carlo Methods in Statistical Physics* (Oxford University Press, 1999).

[4] C. R. Chen and G. I. Makhatadze, "Molecular determinants of temperature dependence of protein volume change upon unfolding," The Journal of Physical Chemistry B **121**, 8300–8310 (2017).